

CS571 Programming Languages

Spring 2026

7. Subroutines and Control



Agenda

- Calling sequence
- Parameter passing modes
- Exception handling

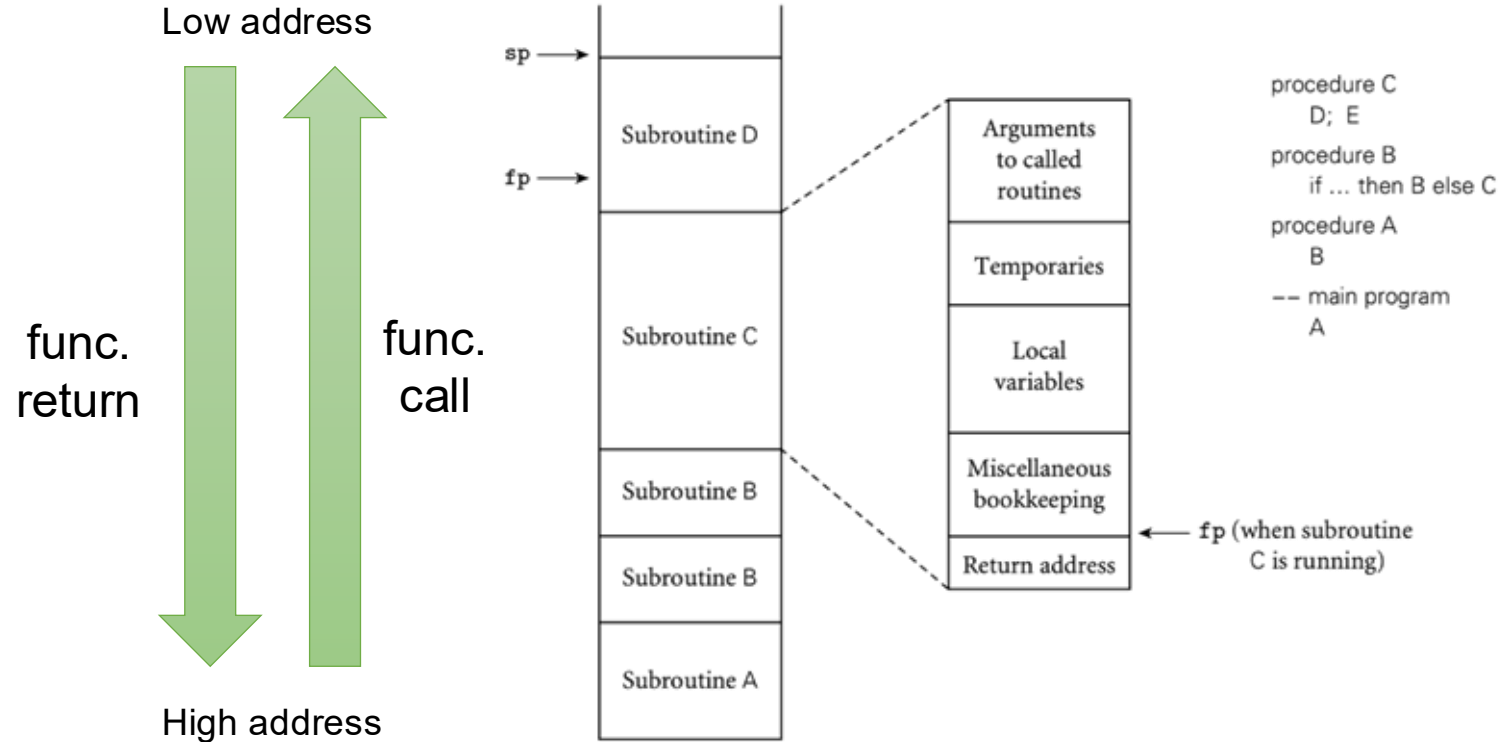
Terminology

- Function (Fortran, Ada): returns a value
- Procedure (Ada), Subroutine (Fortran): returns no value
- C-like language: both are called function
- Method (C++, Java): a function declared inside a class

Caller & Callee

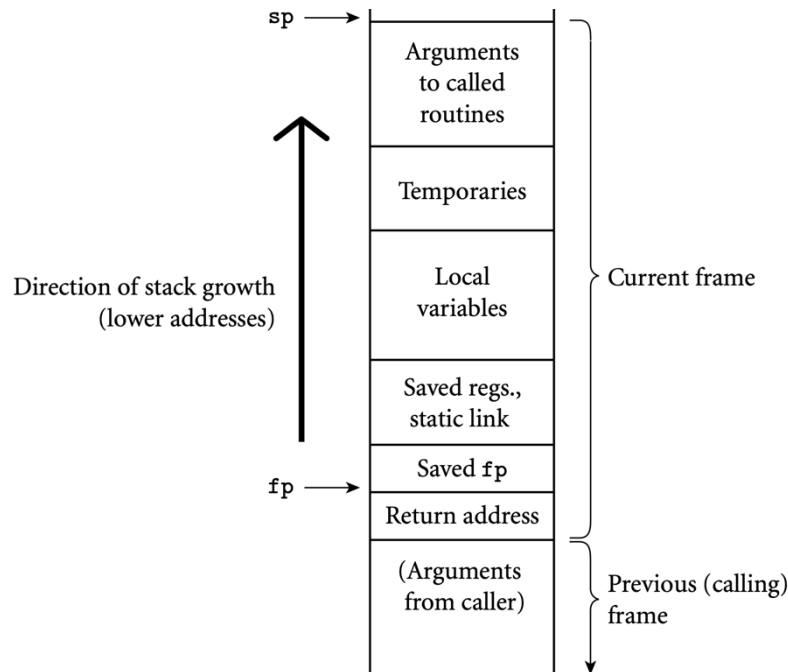
- When function A calls function B
 - A is the Caller
 - B is the Callee
- What needs to be done?
- Who is responsible?

Stack Allocation

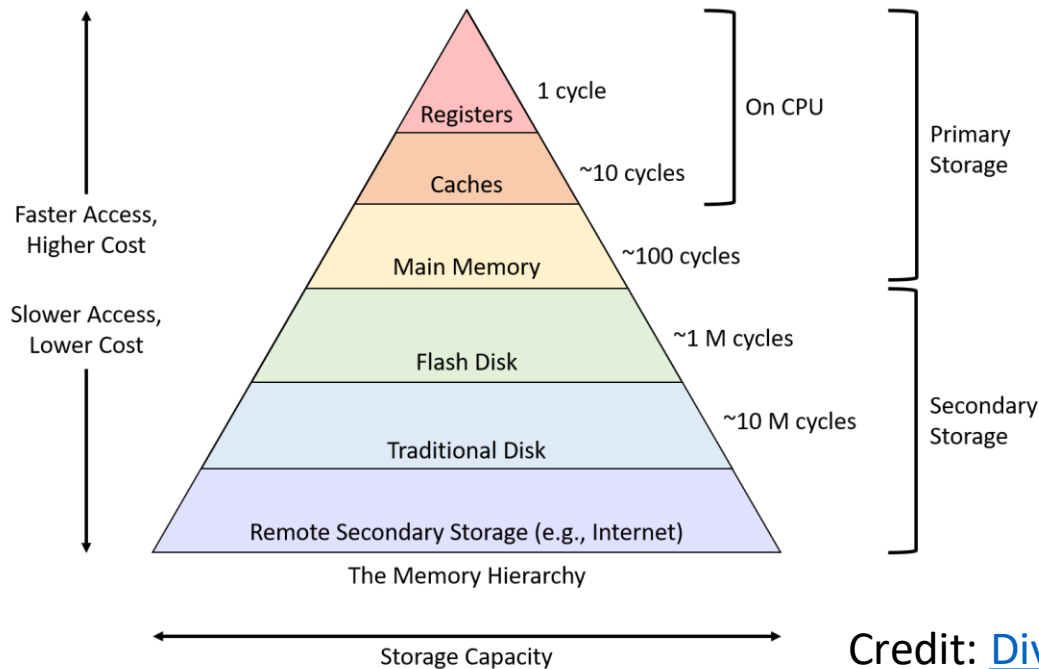


Stack Frame (Activation Record)

- A stack frame contains:
 - Local variables
 - Temporaries
 - Arguments, return values
 - Bookkeeping info
- When is it created? By whom?
- When is it destroyed? By whom?



Memory Hierarchy



Credit: [Dive into Systems](#)

Data Source Latency

Latency Numbers Every Programmer Should Know

■ 1 ns

■ L1 cache reference: 0.5 ns

■ Branch mispredict: 5 ns

■ L2 cache reference: 7 ns

■ Mutex lock/unlock: 25 ns

■ = 100 ns

■ Main memory reference: 100 ns

■ = 1 μ s

■ Compress 1 KB with Zippy: 3 μ s

■ = 10 μ s

■ Send 1 KB over 1 Gbps network: 10 μ s

■ SSD random read (1 Gb/s SSD): 150 μ s

■ Read 1 MB sequentially from memory: 250 μ s

■ Round trip in same datacenter: 500 μ s

■ = 1 ms

■ Read 1 MB sequentially from SSD: 1 ms

■ Disk seek: 10 ms

■ Read 1 MB sequentially from disk: 20 ms

■ Packet roundtrip CA to Netherlands: 150 ms

Source: <https://gist.github.com/2841832>



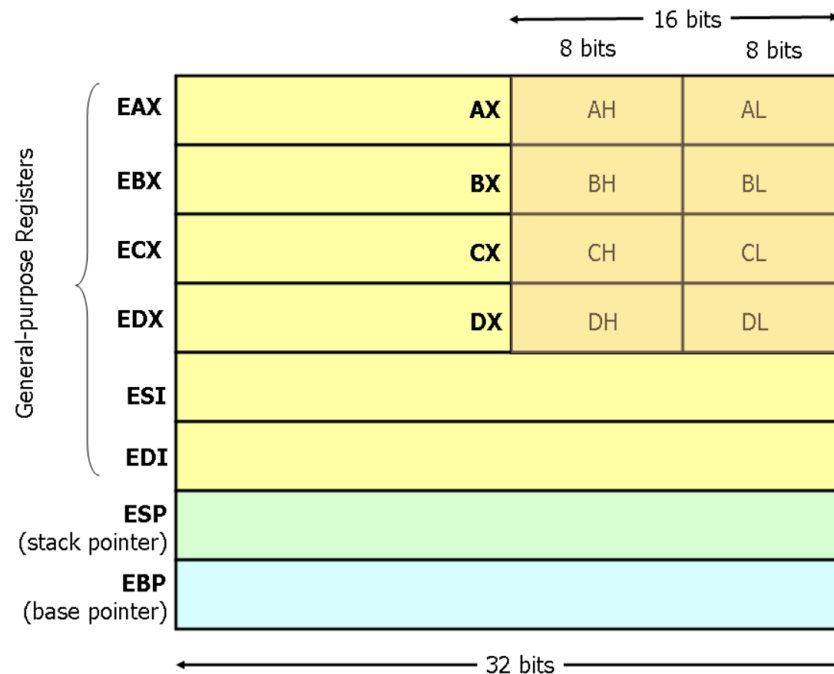
Registers

- How can registers be used?
- When should they be used?
- Who saves registers?

Saving & Restoring Registers

- **Registers are shared**, caller & callee need to save and restore registers
- Ideally: save exactly registers used in caller & callee
- In practice, registers fall into two categories:
 - Caller-saves
 - For transient values not likely needed across calls.
 - Callee-saves
 - For local variables and other long-lived values.

x86 Registers



x86-64 Registers

Register	Conventional use	Low 32-bits	Low 16-bits	Low 8-bits
%rax	Return value, callee-owned	%eax	%ax	%al
%rdi	1st argument, callee-owned	%edi	%di	%dil
%rsi	2nd argument, callee-owned	%esi	%si	%sil
%rdx	3rd argument, callee-owned	%edx	%dx	%dl
%rcx	4th argument, callee-owned	%ecx	%cx	%cl
%r8	5th argument, callee-owned	%r8d	%r8w	%r8b
%r9	6th argument, callee-owned	%r9d	%r9w	%r9b
%r10	Scratch/temporary, callee-owned	%r10d	%r10w	%r10b
%r11	Scratch/temporary, callee-owned	%r11d	%r11w	%r11b

Caller-saved

x86-64 Registers (cont.)

%rsp	Stack pointer, caller-owned	%esp	%sp	%spl
%rbx	Local variable, caller-owned	%ebx	%bx	%bl
%rbp	Local variable, caller-owned	%ebp	%bp	%bpl
%r12	Local variable, caller-owned	%r12d	%r12w	%r12b
%r13	Local variable, caller-owned	%r13d	%r13w	%r13b
%r14	Local variable, caller-owned	%r14d	%r14w	%r14b
%r15	Local variable, caller-owned	%r15d	%r15w	%r15b
%rip	Instruction pointer			
%eflags	Status/condition code bits			

Callee-saved

Often used as
frame pointer

Control Flows

- Program Counter (PC): a register that always contains the memory address of an instruction
 - In x86, it is the `%rip` (or `%eip`) register
- PC always points to the instruction following the one being executed
- PC can be modified by `jump`, `call`, `ret` instructions

A Typical Calling Sequence

- What needs to be done before A calls B?
 - Save dynamic link (current fp stored in `%ebp`)
 - Save other registers
 - Change stack and frame pointer (`%esp`, `%ebp`)
 - Save return address
 - Pass parameters
 - Change program counter



A Typical Calling Sequence

- What needs to be done before B returns to A?
 - Restore saved registers
 - Deallocate stack space of B
 - Pass return values
 - Restore program counter

Review of x86 assembly

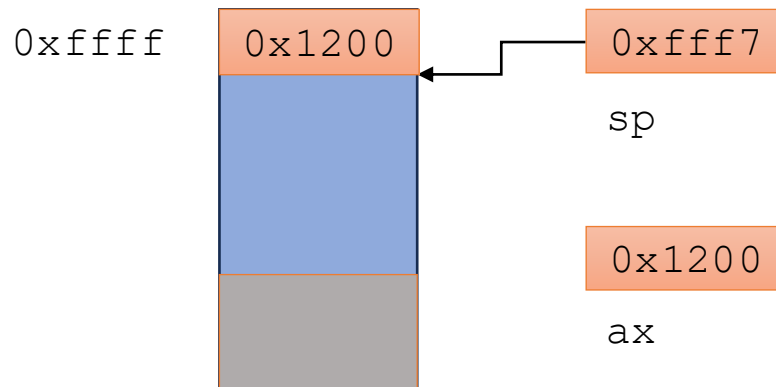
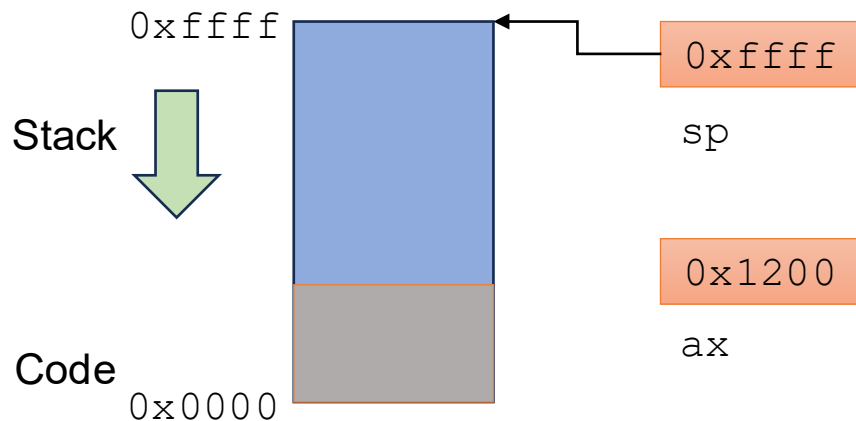
- Mostly two operand instructions
- Unfortunately, two prevalent syntaxes
 - “Intel syntax”: **op dst, src**
 - “AT&T (gcc) syntax”: **op src, dst**
 - We will use AT&T syntax
 - 32-bit AT&T Instructions have **l** suffix, for long
 - 64-bit AT&T Instructions have **q** suffix, for quad

MOV instructions

Assembly	C pseudo-code
<code>movl %eax,%edx</code>	<code>edx = eax;</code>
<code>movl \$0x123, %edx</code>	<code>edx = 0x123;</code>
<code>movl 0x124, %edx</code>	<code>edx = *((int32_t*) 0x124);</code>
<code>movl (%ebx), %edx</code>	<code>edx = *((int32_t*) ebx);</code>
<code>movl 4(%ebx), %edx</code>	<code>edx = *((int32_t*) (ebx+4));</code>

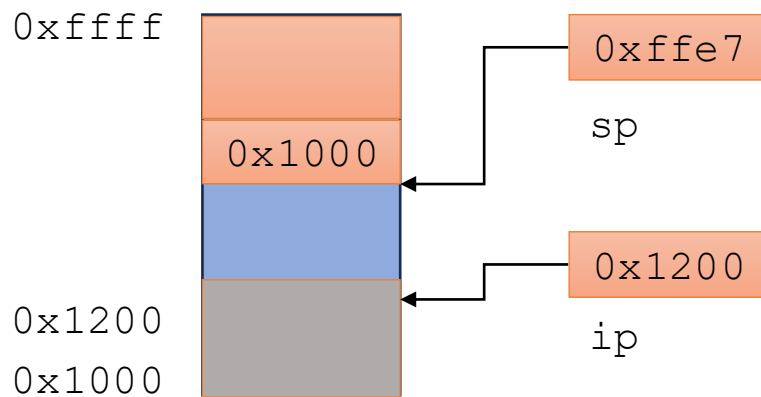
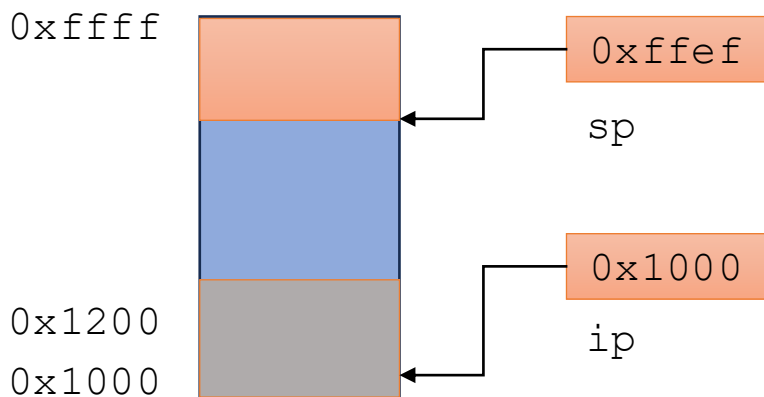
Stack instructions

Assembly	C pseudo-code
<code>pushq %rax</code>	<code>subq \$8,%rsp</code> <code>movq %rax, (%rsp)</code>
<code>popq %rax</code>	<code>movq (%rsp), %rax</code> <code>addq \$8,%rsp</code>



Function call/ret

Assembly	C pseudo-code
<code>callq \$0x1200</code>	<code>pushq %rip</code> <code>movq \$0x1200,%rip</code>
<code>retq</code>	<code>popq %rip</code>



A Typical Calling Sequence

1. The Caller A: prep before calling B
2. The Callee B: prologue before running B
3. The Callee B: epilogue after completing B before returning to A
4. The Caller A: clean up after calls to B

Calling Sequence Example

```

int f(int x) {
    int y = 2;
    return x + y;
}

int main(void) {
    int y = 10;
    y = f(3);
    return y;
}

```



```

0000000100003f50 <_f>:
100003f50: 55                pushq   %rbp
100003f51: 48 89 e5          movq    %rsp, %rbp
100003f54: 89 7d fc          movl    %edi, -4(%rbp)
100003f57: c7 45 f8 02 00 00 00 movl    $2, -8(%rbp)
100003f5e: 8b 45 fc          movl    -4(%rbp), %eax
100003f61: 03 45 f8          addl    -8(%rbp), %eax
100003f64: 5d                popq    %rbp
100003f65: c3                retq
100003f66: 66 2e 0f 1f 84 00 00 00 00 nopw    %cs:(%rax,%rax)

0000000100003f70 <_main>:
100003f70: 55                pushq   %rbp
100003f71: 48 89 e5          movq    %rsp, %rbp
100003f74: 48 83 ec 10       subq    $16, %rsp
100003f78: c7 45 fc 00 00 00 00 movl    $0, -4(%rbp)
100003f7f: c7 45 f8 0a 00 00 00 movl    $10, -8(%rbp)
100003f86: bf 03 00 00 00    movl    $3, %edi
100003f8b: e8 c0 ff ff ff    callq   0x100003f50 <_f>
100003f90: 89 45 f8          movl    %eax, -8(%rbp)
100003f93: 8b 45 f8          movl    -8(%rbp), %eax
100003f96: 48 83 c4 10       addq    $16, %rsp
100003f9a: 5d                popq    %rbp
100003f9b: c3                retq

```

Compiled with clang to x86

```

$clang -arch x86_64 call.c
$objdump -d a.out

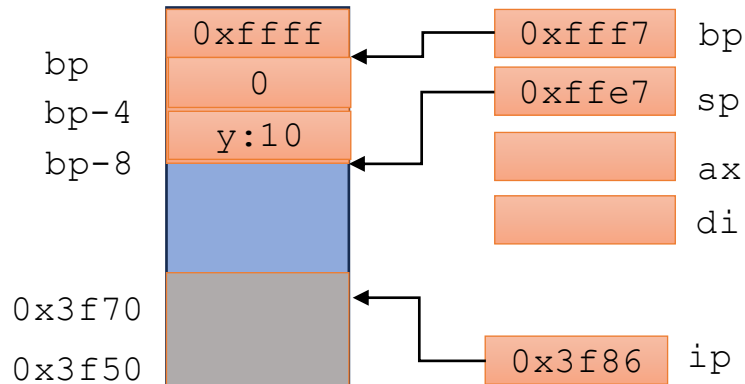
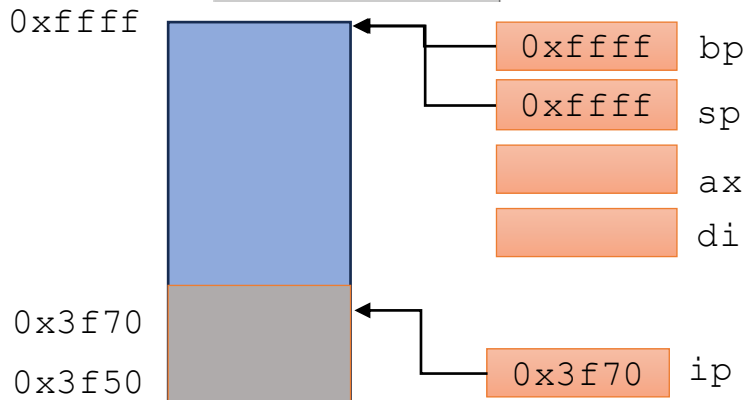
```

Before the call

```
int main(void) {
    int y = 10;
    y = f(3);
    return y;
}
```

0000000100003f70 <_main>:

100003f70: 55	pushq %rbp
100003f71: 48 89 e5	movq %rsp, %rbp
100003f74: 48 83 ec 10	subq \$16, %rsp
100003f78: c7 45 fc 00 00 00 00	movl \$0, -4(%rbp)
100003f7f: c7 45 f8 0a 00 00 00	movl \$10, -8(%rbp)
100003f86: bf 03 00 00 00	movl \$3, %edi
100003f8b: e8 c0 ff ff ff	callq 0x100003f50 <_f>
100003f90: 89 45 f8	movl %eax, -8(%rbp)
100003f93: 8b 45 f8	movl -8(%rbp), %eax
100003f96: 48 83 c4 10	addq \$16, %rsp
100003f9a: 5d	popq %rbp
100003f9b: c3	retq



Calling Sequence 1

```
int main(void) {
    int y = 10;
    y = f(3);
    return y;
}
```

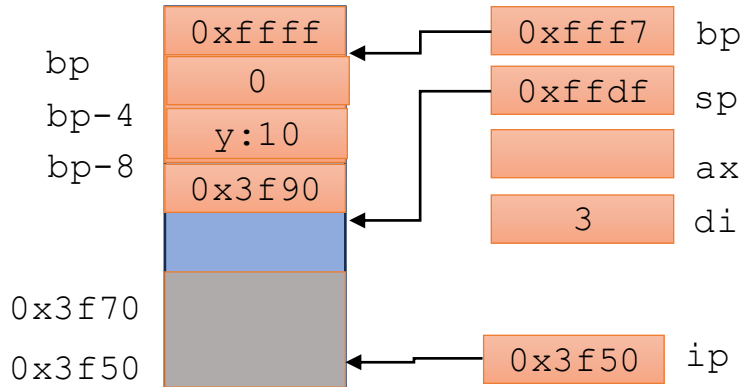
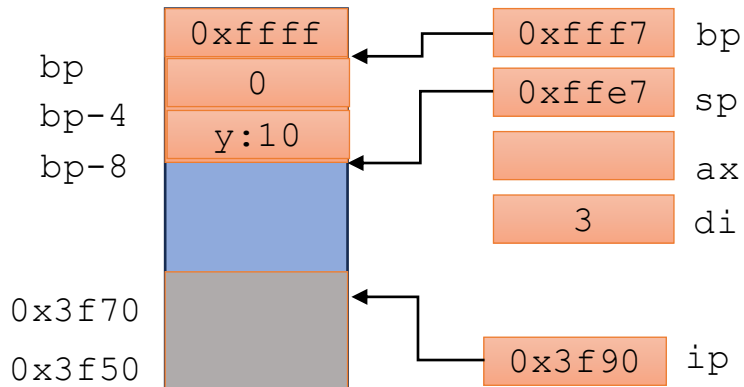
00000000100003f70 <_main>:

```
100003f70: 55
100003f71: 48 89 e5
100003f74: 48 83 ec 10
100003f78: c7 45 fc 00 00 00 00
100003f7f: c7 45 f8 0a 00 00 00
100003f86: bf 03 00 00 00
100003f8b: e8 c0 ff ff ff
100003f90: 89 45 f8
100003f93: 8b 45 f8
100003f96: 48 83 c4 10
100003f9a: 5d
100003f9b: c3
```

```
pushq    %rbp
movq     %rsp, %rbp
subq     $16, %rsp
movl     $0, -4(%rbp)
movl     $10, -8(%rbp)
movl     $3, %edi
callq    0x100003f50 <_f>
movl     %eax, -8(%rbp)
movl     -8(%rbp), %eax
addq     $16, %rsp
popq     %rbp
retq
```

```
pushq %rip
movq $0x3f50,%rip
```

ret addr →



Calling Sequence 1

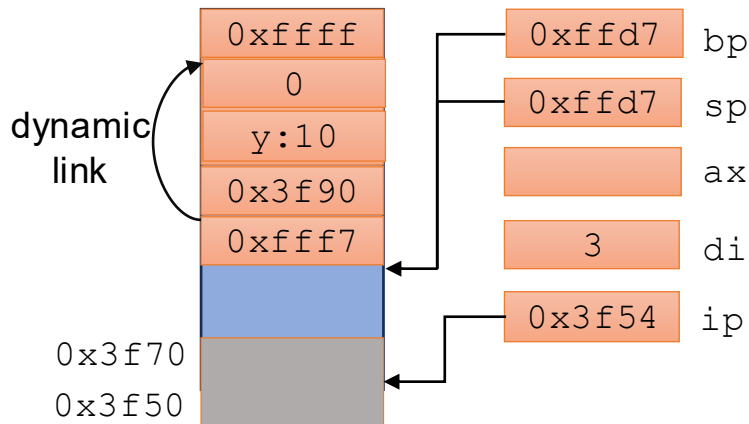
The Caller:

- Save any caller-saved registers that may be needed after the call
- Compute the values of arguments and moves them into the stack or registers
- Computes the static link (if the language has nested subroutines), and passes it as an extra, hidden argument
- Use a special subroutine call instruction to jump to the subroutine, simultaneously passing the return address on the stack or in a register

```
int main(void) {  
    int y = 10;  
    y = f(3);  
    return y;  
}
```

```
pushq    %rbp  
movq     %rsp, %rbp  
subq     $16, %rsp  
movl     $0, -4(%rbp)  
movl     $10, -8(%rbp)  
movl     $3, %edi  
callq    0x100003f50 <_f>  
movl     %eax, -8(%rbp)  
movl     -8(%rbp), %eax  
addq     $16, %rsp  
popq     %rbp  
retq
```

0000000100003f50 <_f>:			
100003f50:	55		pushq %rbp
100003f51:	48 89 e5		movq %rsp, %rbp
100003f54:	89 7d fc		movl %edi, -4(%rbp)
100003f57:	c7 45 f8 02 00 00 00		movl \$2, -8(%rbp)
100003f5e:	8b 45 fc		movl -4(%rbp), %eax
100003f61:	03 45 f8		addl -8(%rbp), %eax
100003f64:	5d		popq %rbp
100003f65:	c3		retq
100003f66:	66 2e 0f 1f 84 00 00 00 00 00		nopw%cs:(%rax,%rax)



Calling Sequence 2

Prologue in Callee

- allocates a frame by subtracting an appropriate constant from the **sp**
- saves the old frame pointer into the stack, and updates it to point to the newly allocated frame
- saves any callee-saved register that may be overwritten by the current routine

```
int f(int x) {  
    int y = 2;  
    return x + y;  
}
```

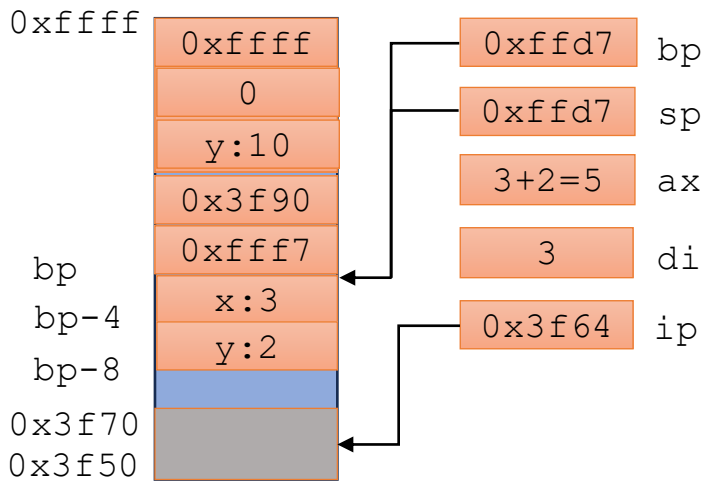
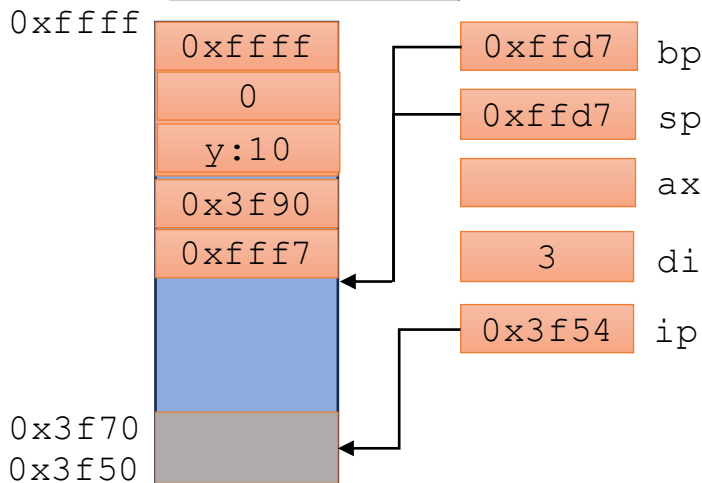


```
pushq    %rbp  
movq     %rsp, %rbp  
movl     %edi, -4(%rbp)  
movl     $2, -8(%rbp)  
movl     -4(%rbp), %eax  
addl     -8(%rbp), %eax  
popq     %rbp  
retq  
  
nopw%cs: (%rax,%rax)
```

Execution of Callee

```
int f(int x) {
    int y = 2;
    return x + y;
}
```

00000000100003f50 <_f>:	
100003f50: 55	pushq %rbp
100003f51: 48 89 e5	movq %rsp, %rbp
100003f54: 89 7d fc	movl %edi, -4(%rbp)
100003f57: c7 45 f8 02 00 00 00	movl \$2, -8(%rbp)
100003f5e: 8b 45 fc	movl -4(%rbp), %eax
100003f61: 03 45 f8	addl -8(%rbp), %eax
100003f64: 5d	popq %rbp
100003f65: c3	retq
100003f66: 66 2e 0f 1f 84 00 00 00 00	nopw%cs:(%rax,%rax)



Calling Sequence 3

```
int f(int x) {
    int y = 2;
    return x + y;
}
```

00000000100003f50 <_f>:

100003f50: 55

100003f51: 48 89 e5

100003f54: 89 7d fc

100003f57: c7 45 f8 02 00 00 00

100003f5e: 8b 45 fc

100003f61: 03 45 f8

100003f64: 5d

100003f65: c3

100003f66: 66 2e 0f 1f 84 00 00 00 00 00

pushq %rbp

movq %rsp, %rbp

movl %edi, -4(%rbp)

movl \$2, -8(%rbp)

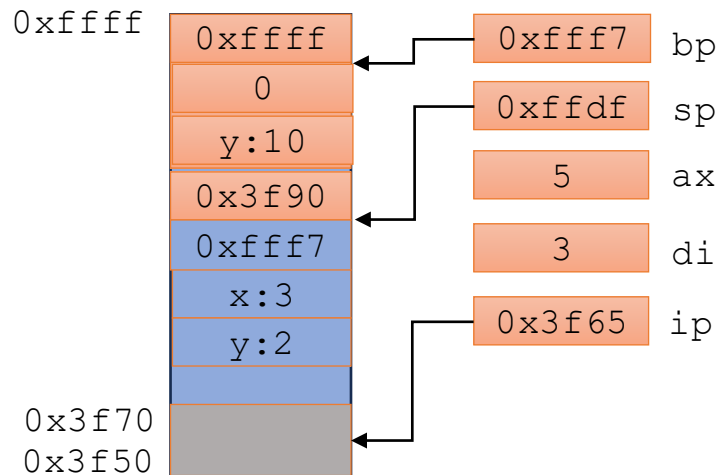
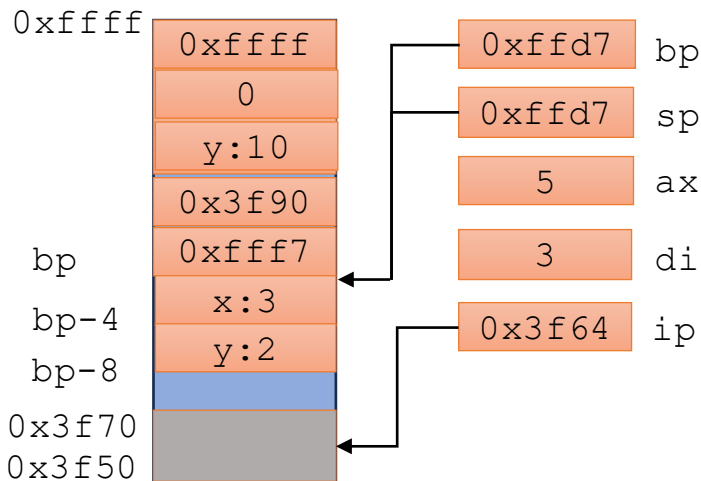
movl -4(%rbp), %eax

addl -8(%rbp), %eax

popq %rbp

retq

nopw%cs:(%rax,%rax)



Calling Sequence 3

```
int f(int x) {
    int y = 2;
    return x + y;
}
```

00000000100003f50 <_f>:

100003f50: 55

100003f51: 48 89 e5

100003f54: 89 7d fc

100003f57: c7 45 f8 02 00 00 00

100003f5e: 8b 45 fc

100003f61: 03 45 f8

100003f64: 5d

100003f65: c3

100003f66: 66 2e 0f 1f 84 00 00 00 00

pushq %rbp

movq %rsp, %rbp

movl %edi, -4(%rbp)

movl \$2, -8(%rbp)

movl -4(%rbp), %eax

addl -8(%rbp), %eax

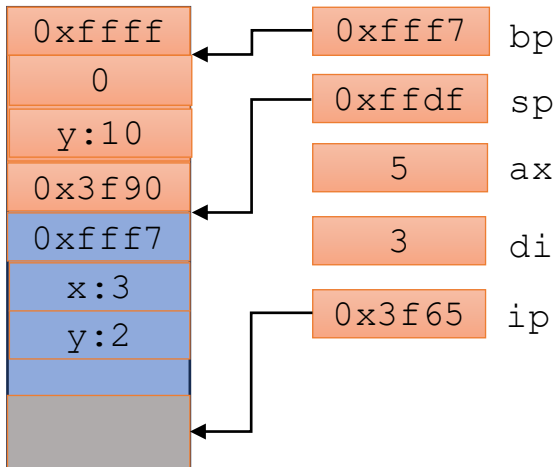
popq %rbp

retq

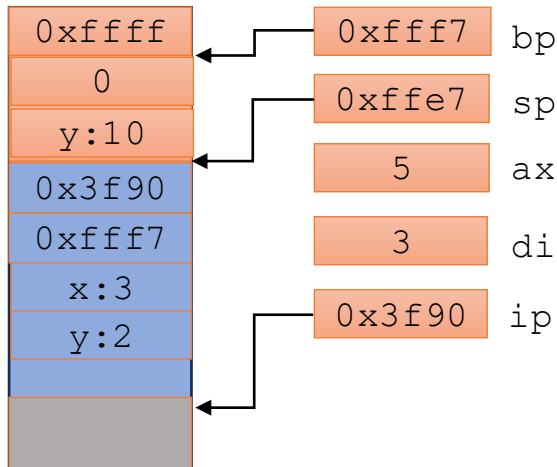
popq %rip

nopw%cs:(%rax,%rax)

0xffff



0xffff



Calling Sequence 3

Epilogue in Callee

- moves the return value into a register (e.g. **eax**) or a reserved location in the stack
- restores callee-saves registers if needed
- restores the **fp** and the **sp**
- jumps back to the return address

```
int f(int x) {
    int y = 2;
    return x + y;
}
```

00000000100003f50 <_f>:	
100003f50: 55	pushq %rbp
100003f51: 48 89 e5	movq %rsp, %rbp
100003f54: 89 7d fc	movl %edi, -4(%rbp)
100003f57: c7 45 f8 02 00 00 00	movl \$2, -8(%rbp)
100003f5e: 8b 45 fc	movl -4(%rbp), %eax
100003f61: 03 45 f8	addl -8(%rbp), %eax
100003f64: 5d	popq %rbp
100003f65: c3	retq
100003f66: 66 2e 0f 1f 84 00 00 00 00 00	nopw%cs:(%rax,%rax)

Calling Sequence 4

```
int main(void) {
    int y = 10;
    y = f(3);
    return y;
}
```

```
□
```

```
~
```

```
~
```

```
~
```

```
~
```

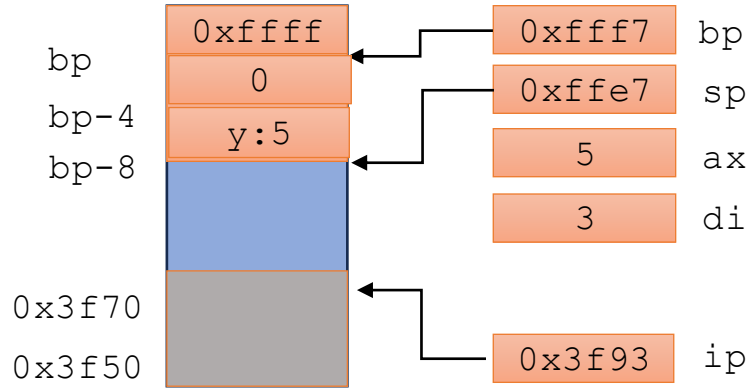
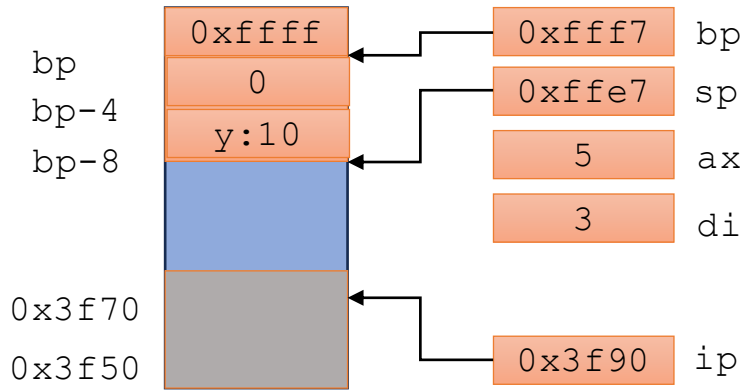
```
~
```

```
~
```

```
0000000100003f70 <_main>:
```

```
100003f70: 55
100003f71: 48 89 e5
100003f74: 48 83 ec 10
100003f78: c7 45 fc 00 00 00 00
100003f7f: c7 45 f8 0a 00 00 00
100003f86: bf 03 00 00 00
100003f8b: e8 c0 ff ff ff
100003f90: 89 45 f8
100003f93: 8b 45 f8
100003f96: 48 83 c4 10
100003f9a: 5d
100003f9b: c3
```

```
pushq    %rbp
movq     %rsp, %rbp
subq     $16, %rsp
movl     $0, -4(%rbp)
movl     $10, -8(%rbp)
movl     $3, %edi
callq    0x100003f50 <_f>
movl     %eax, -8(%rbp)
movl     -8(%rbp), %eax
addq     $16, %rsp
popq     %rbp
retq
```



A Typical Calling Sequence 4

The Caller:

- moves the return value to wherever it is needed
- restores caller-saves registers if needed

```
int main(void) {  
    int y = 10;  
    y = f(3);  
    return y;  
}
```

```
pushq    %rbp  
movq     %rsp, %rbp  
subq     $16, %rsp  
movl     $0, -4(%rbp)  
movl     $10, -8(%rbp)  
movl     $3, %edi  
callq    0x100003f50 <_f>  
movl     %eax, -8(%rbp)  
movl     -8(%rbp), %eax  
addq     $16, %rsp  
popq     %rbp  
retq
```

Example

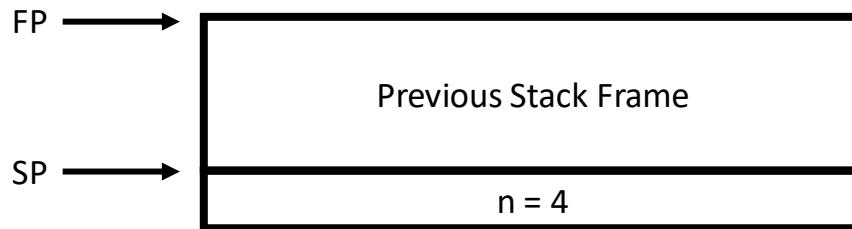
```
int fact(int n)
{
    int result = 1;
    while (n>0)
    {
        result = result * n;
        n = n-1
    }
    return result;
}
```



→ fact(4)

Example

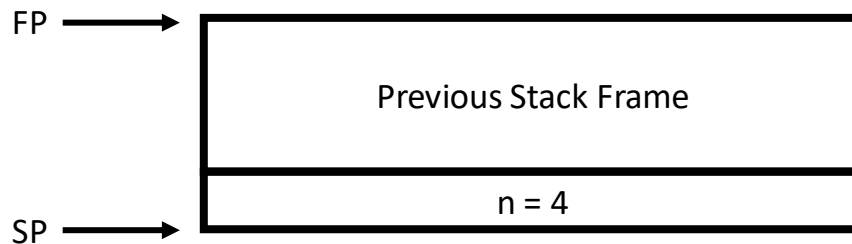
```
int fact(int n)
{
    int result = 1;
    while (n>0)
    {
        result = result * n;
        n = n-1
    }
    return result;
}
```



→ fact(4)

Example

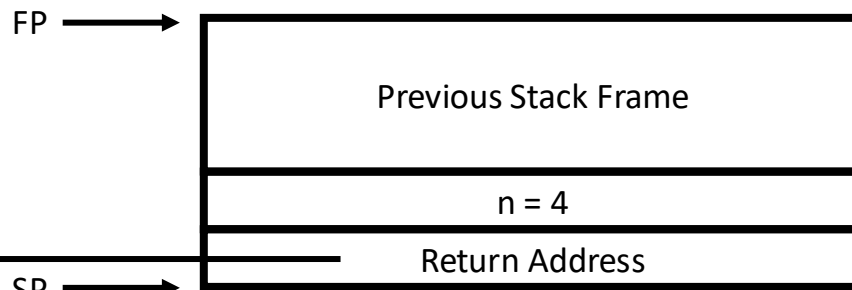
```
int fact(int n)
{
    int result = 1;
    while (n>0)
    {
        result = result * n;
        n = n-1
    }
    return result;
}
```



→ `fact(4)`

Example

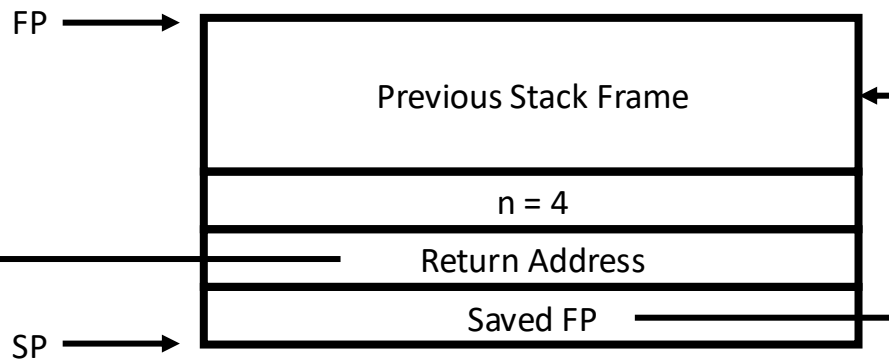
```
int fact(int n)
{
    int result = 1;
    while (n>0)
    {
        result = result * n;
        n = n-1
    }
    return result;
}
```



fact(4)

Example

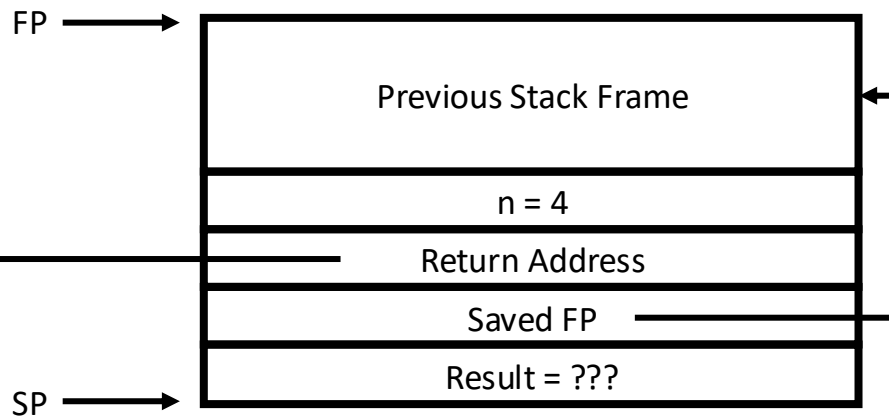
```
int fact(int n)
{
    int result = 1;
    while (n>0)
    {
        result = result * n;
        n = n-1
    }
    return result;
}
```



fact(4)

Example

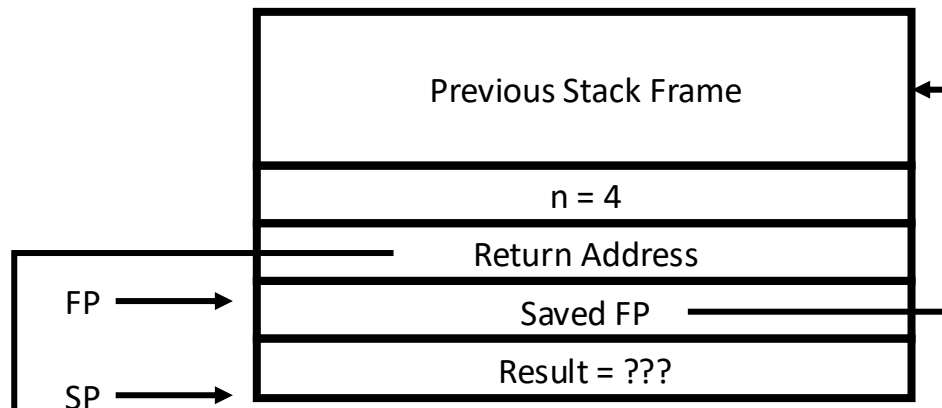
```
int fact(int n)
{
    int result = 1;
    while (n>0)
    {
        result = result * n;
        n = n-1
    }
    return result;
}
```



fact(4)

Example

```
int fact(int n)
{
    int result = 1;
    while (n>0)
    {
        result = result * n;
        n = n-1
    }
    return result;
}
```

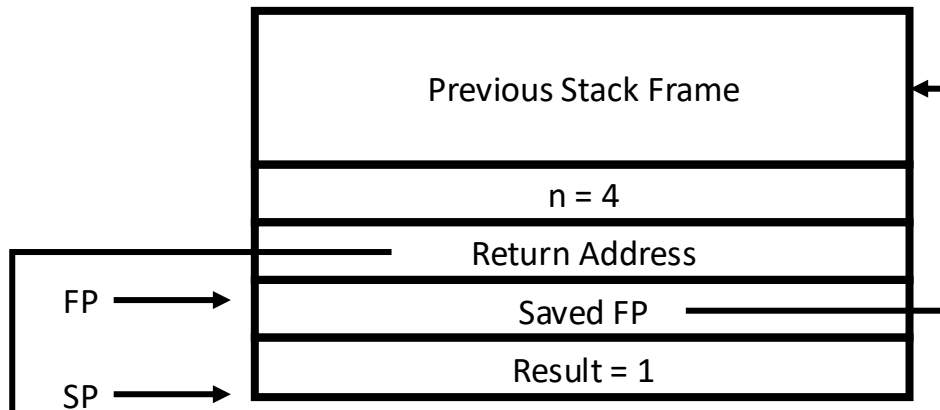


→ `fact(4)` ←

Example

```
int fact(int n)
{
    → int result = 1;
    while (n>0)
    {
        result = result * n;
        n = n-1
    }
    return result;
}
```

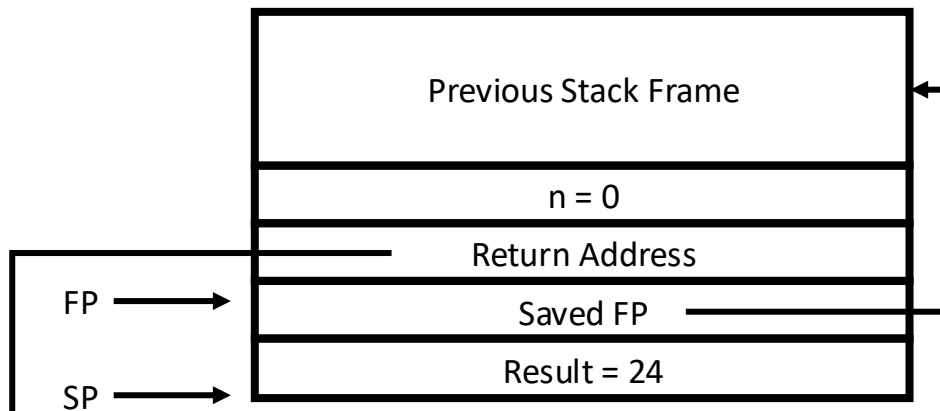
fact(4) ←



Example

```
int fact(int n)
{
    int result = 1;
    while (n>0)
    {
        result = result * n;
        n = n-1
    }
    return result;
}
```

fact(4) ←

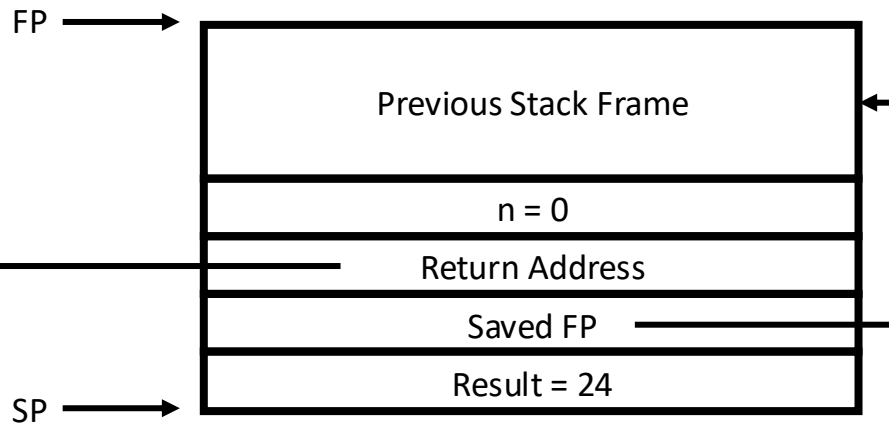


Example

```
int fact(int n)
{
    int result = 1;
    while (n>0)
    {
        result = result * n;
        n = n-1
    }
    return result;
}
```

→ return result;

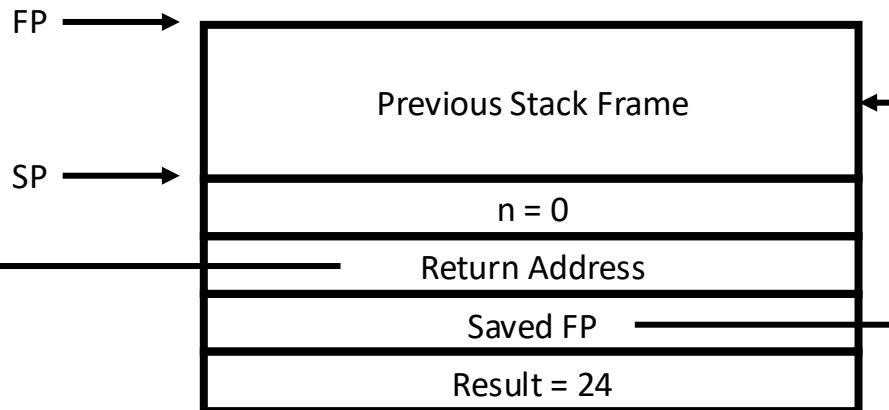
fact(4) ←



Example

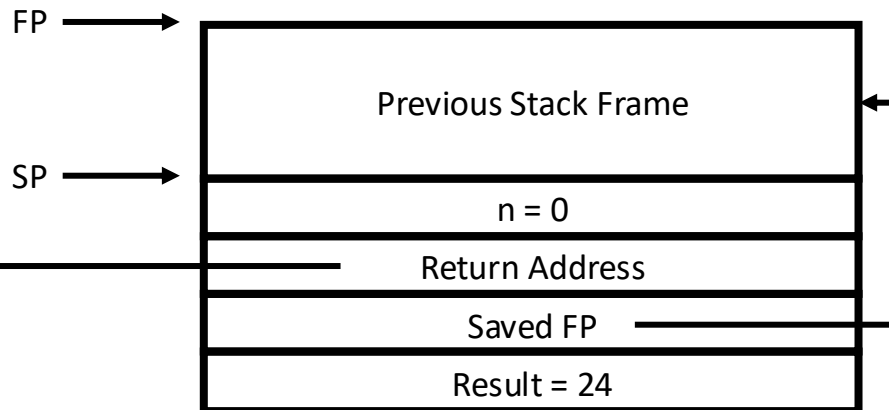
```
int fact(int n)
{
    int result = 1;
    while (n>0)
    {
        result = result * n;
        n = n-1
    }
    return result;
}
```

fact(4) ←



Example

```
int fact(int n)
{
    int result = 1;
    while (n>0)
    {
        result = result * n;
        n = n-1
    }
    return result;
}
```



fact(4)

Example

```
int fact(int n)
{
    int result = 1;
    while (n>0)
    {
        result = result * n;
        n = n-1
    }
    return result;
}
```



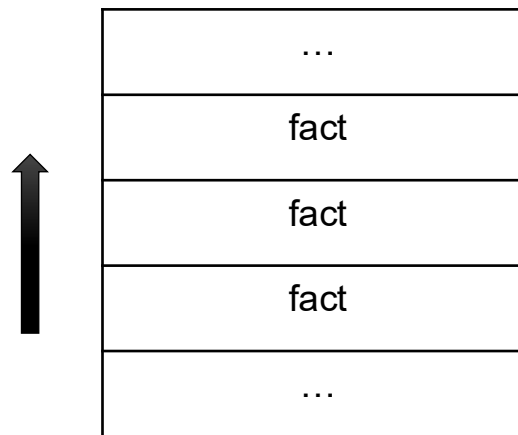
→ fact(4)

Function Calls are Costly

- Function call is **space and time consuming**
 - Space: each active function requires one frame (activation record) on stack
 - Time: instructions are added by the compiler to set up and clean up function calls (calling sequence)

Recursion

```
int fact(int n) {  
    if (n==1)  
        return 1;  
    else  
        return n*fact(n-1);  
}
```

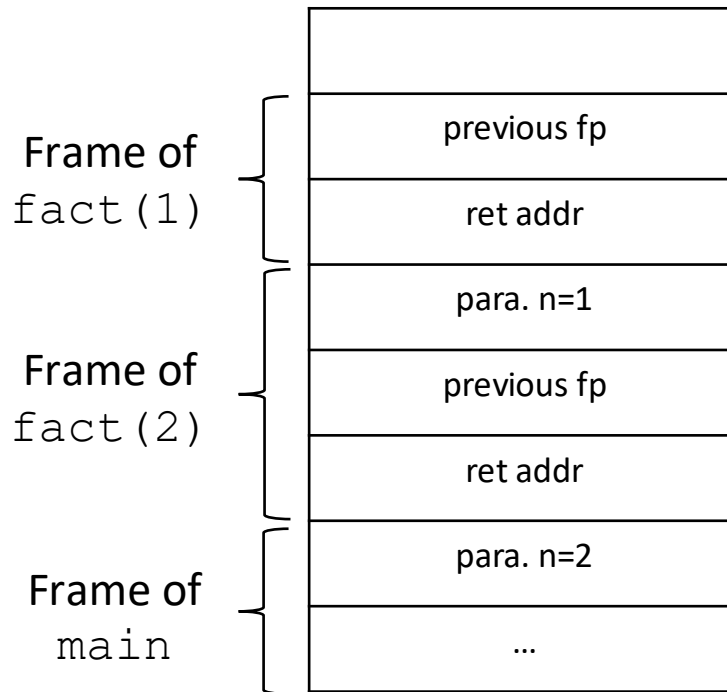


During execution, each function call has one frame (activation record) on the stack

Recursion

```
int fact(int n) {  
    if (n==1)  
        return 1;  
    else  
        return n*fact(n-1);  
}
```

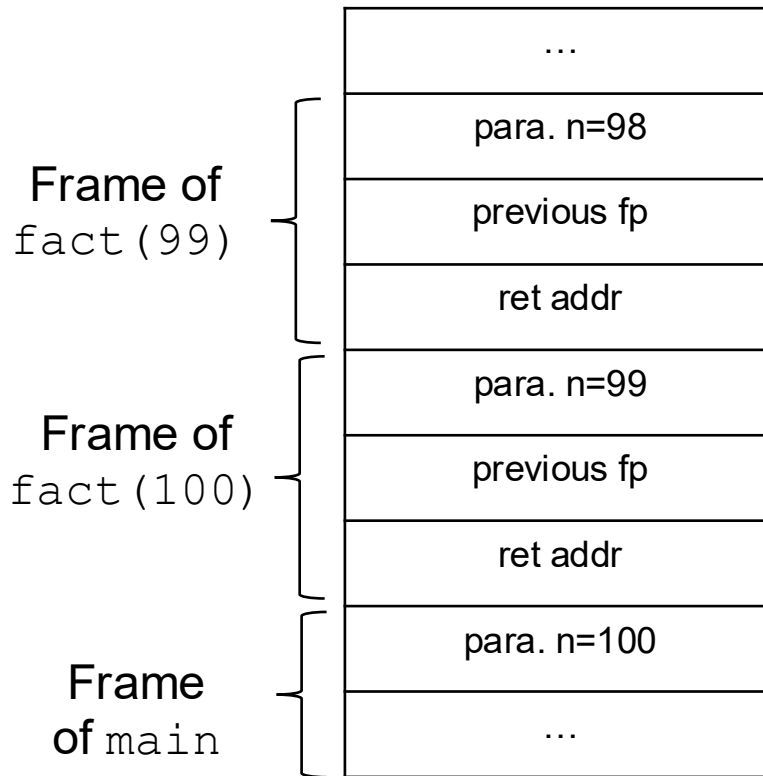
Calling fact(2) in main



Recursion

```
int fact(int n) {  
    if (n==1)  
        return 1;  
    else  
        return n*fact(n-1);  
}
```

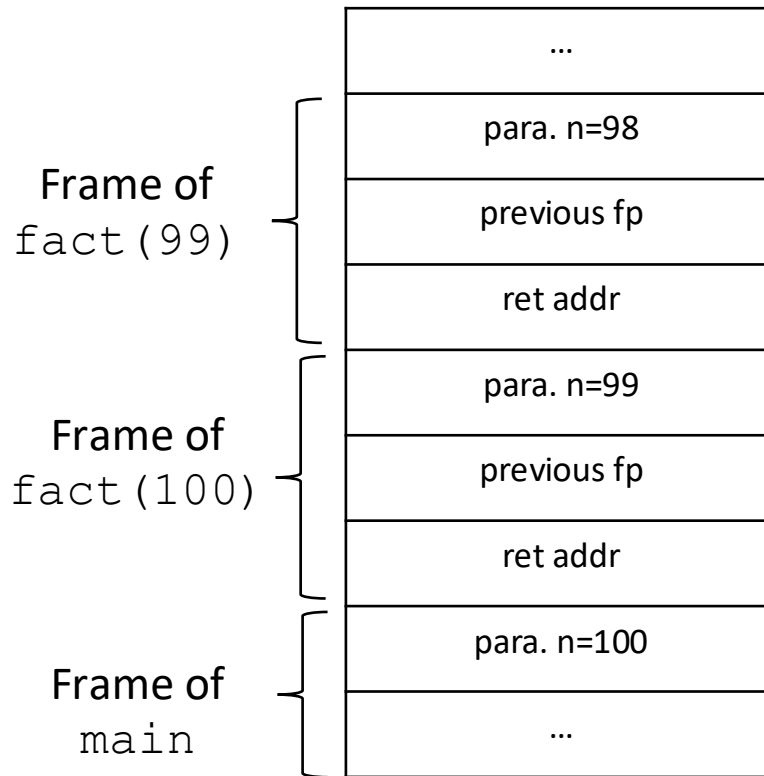
- **Calling** `fact(100)` in `main`
 - Consumes memory
 - Wastes time on calling sequence



Recursion

```
int fact(int n) {  
    if (n==1)  
        return 1;  
    else  
        return n*fact(n-1);  
}
```

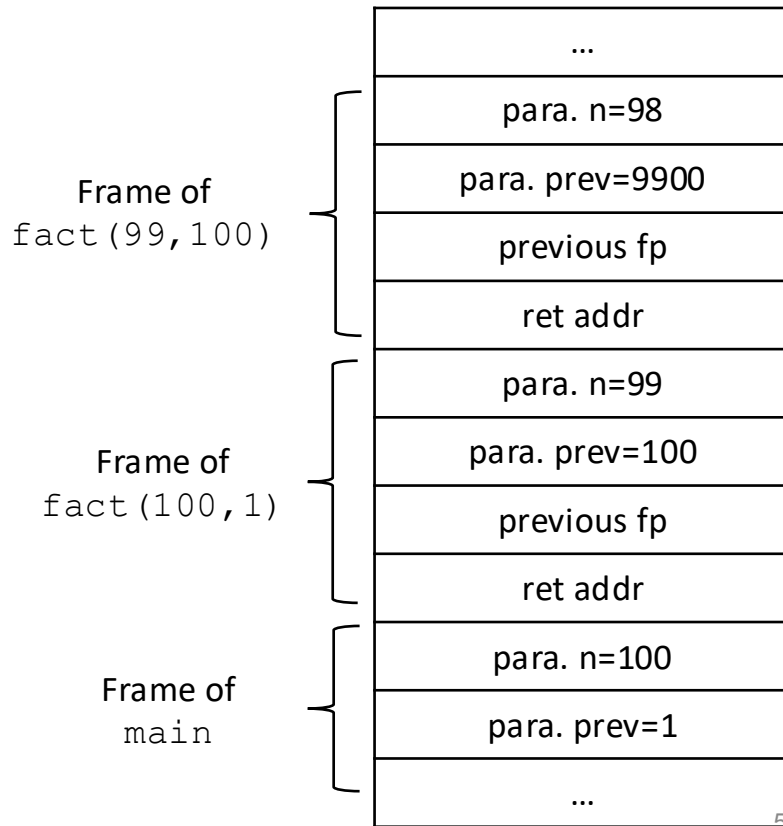
- Can we destroy the frame for `fact(100)` before going into `fact(99)`?



A Different Implementation

```
int fact(int n, int prev) {  
    if (n==1)  
        return prev ;  
    else  
        return fact(n-1,prev*n);  
}
```

- Calling `fact(100, 1)` in main
- Can we destroy the frame for `fact(100, 1)` before going into `fact(99, 100)`?



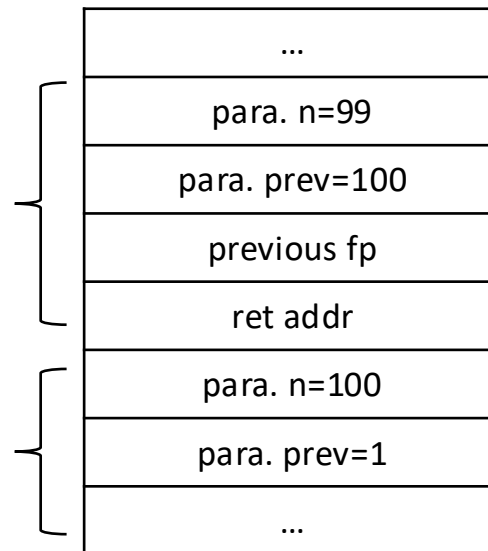
A Different Implementation

```
int fact(int n, int prev) {  
    if (n==1)  
        return prev ;  
    else  
        return fact(n-1,prev*n);  
}
```

- Calling `fact(100, 1)` in main
- What change enables the more efficient implementation?

Reused frame

Frame of
main



Tail-Recursive Functions

```
int fact(int n, int prev) {  
    if (n==1)  
        return prev ;  
    else  
        return fact(n-1,prev*n);  
}
```

- Recursion only occur at the end of a function: no computation after recursive call
- The current activation record before recursive call is useless!

Tail-Recursive Functions

Tail-recursive functions are equivalent to loops

```
int fact(int n, int prev){  
    if (n==1)  
        return prev ;  
    else  
        return fact(n-1,prev*n);  
}
```

```
int fact(int n, int prev){  
    while (true) {  
        if (n==1)  
            return prev ;  
        else {  
            prev = prev*n;  
            n--;  
        }  
    }  
}
```

Is this function tail recursive?

```
int search(int[] a, int k, int fst, int lst){  
    if (fst==lst)  
        return (a[fst]==k? fst:-1);  
    int m=(fst+lst)/2;  
    if (k<=a[m])  
        return search(a, k, fst, m);  
    else  
        return search(a, k, m+1, lst);  
}
```


Using Loops

```
int search(int[] a, int k, int fst, int lst){  
    while (true) {  
        if (fst==lst)  
            return (a[fst]==k? fst:-1);  
        int m=(fst+lst)/2;  
        if (k<=a[m])  
            lst=m;  
        else  
            fst=m+1;  
    }  
}
```

Tail-Recursive Functions to Loops

- In C and Java, the loop version is more efficient, though some compilers will automatically generate more efficient code
- In most functional languages (e.g., Scheme, Ocaml), the compiler automatically generate code as efficient as the loop version



Agenda

- Calling sequence
- Parameter passing modes
- Exception handling

Parameter Passing

How caller communicates with callee

- Formal parameters: names in the declaration of a function
- Actual parameters: variables/expressions passed to a function

```
int foo (int x) {  
    return x+2;  
};  
foo (3+5);
```

Formal para.

Actual para.



Parameter Modes

- Call-By-Value (CBV)
- Call-By-Value-Return (CBVR)
- Call-By-Reference (CBR)
- Call-By-Name (CBN)

L-value vs R-value

- L-value

- Refers to object that persists beyond a single expression
- Can occur on the LHS of an assignment statement
- Examples: `x`, `a[i]`

- R-value

- A temporary value that does not persist beyond the expression that uses it
- Can occur on the RHS of an assignment statement
- Examples: `x + 1`, `x < 2`, literals like `42` or `"hello"`, compile-time constants

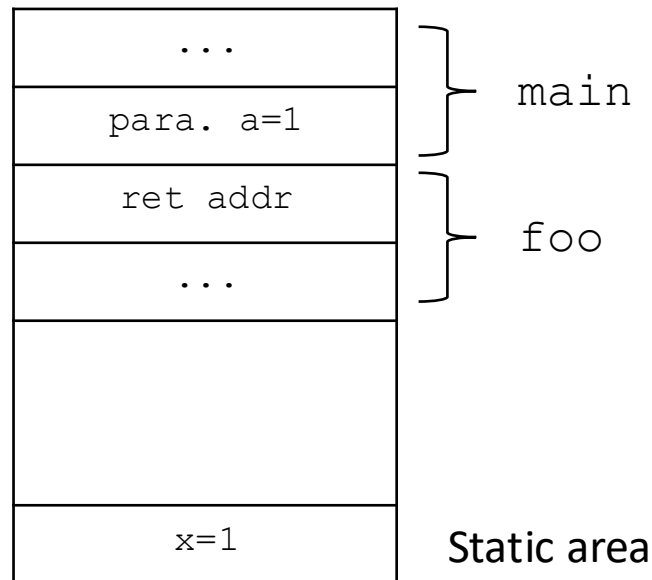
- L-values have an address; R-values need not have one

Call-By-Value

- Calling mechanism
 - Arguments are evaluated to their values
 - Memory or registers allocated for arguments on AR
 - Argument **values** copied to AR
 - AR destroyed when callee returns

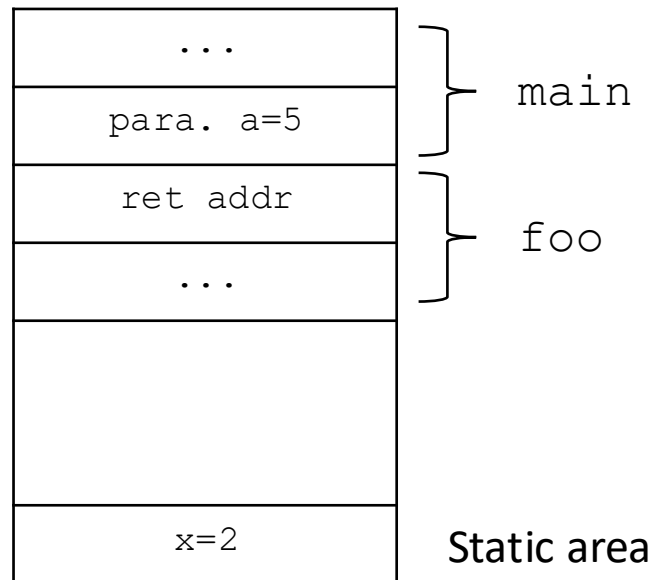
Call-By-Value

```
int x = 1;
int foo (int a) {
    x = 2;
    print (a);
    a = 5;
    return x+a;
}
void main() {
    foo(x); //result?
    print(x); //result?
}
```



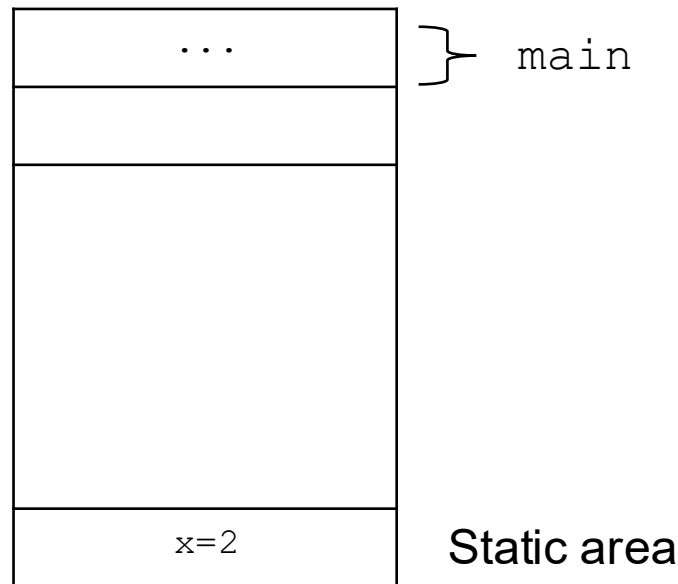
Call-By-Value

```
int x = 1;
int foo (int a) {
    x = 2;
    print (a);
    a = 5;
    return x+a;
}
void main() {
    foo(x); //result?
    print(x); //result?
}
```



Call-By-Value

```
int x = 1;
int foo (int a) {
    x = 2;
    print (a);
    a = 5;
    return x+a;
}
void main() {
    foo(x); //result?
    print(x); //result?
}
```



Call-By-Value

- Formal & actual params have **separate memory**
 - Their values may diverge
- Characteristics
 - Actual parameters may not directly be changed in callee (unless pass in pointers)
 - Arguments can be complex expressions
 - Simple and intuitive (less error-prone)



Call-By-Value: Performance

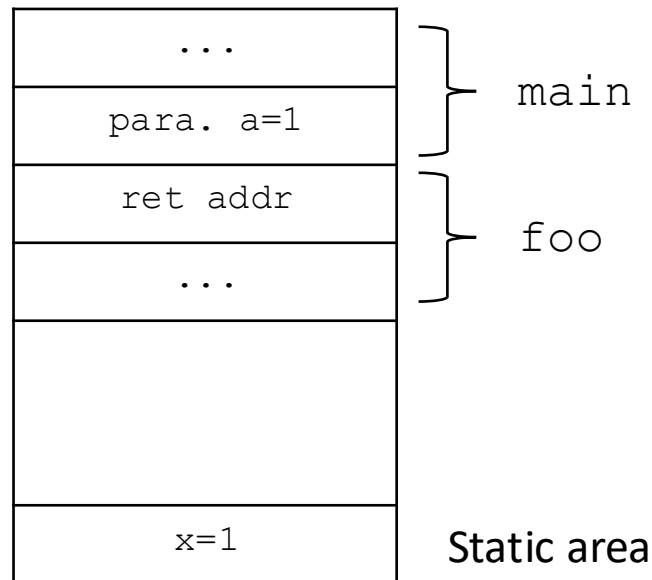
- Primitive types: cost per parameter is small
- Arrays, records, structures: copying value could be slow
- C: call-by-value is the default mode
- Java: only primitive types use call-by-value

Call-By-Value-Return (In-Out)

- Calling mechanism
 - Arguments are evaluated to their values
 - Memory or registers allocated for arguments on AR
 - Argument values stored in AR
 - Before callee returns, AR values copied back to actual arguments
 - AR destroyed when callee returns

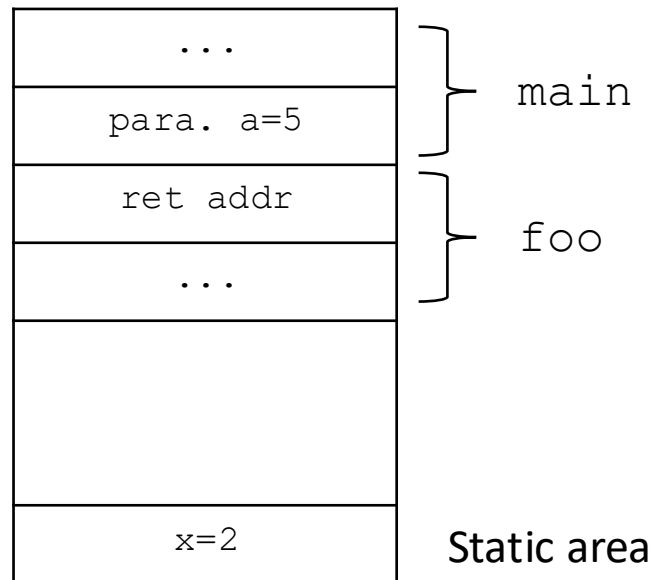
Call-By-Value-Return

```
int x = 1;
int foo (int a) {
    x = 2;
    print (a);
    a = 5;
    return x+a;
}
void main() {
    foo(x); //result?
    print(x); //result?
}
```



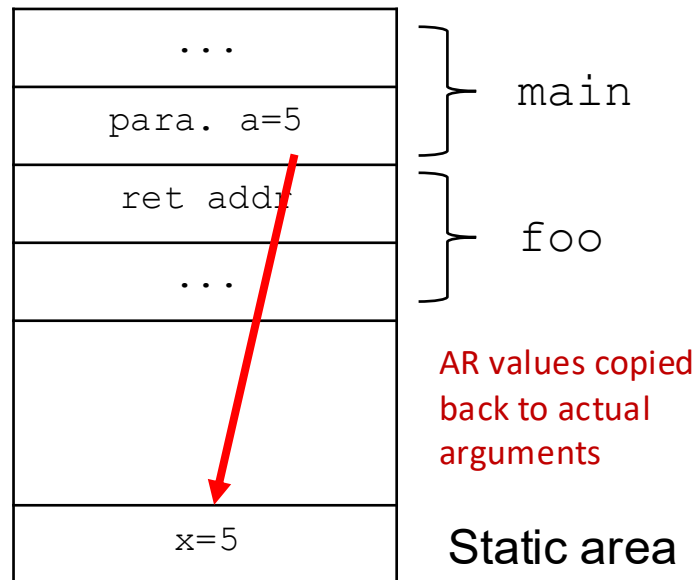
Call-By-Value

```
int x = 1;
int foo (int a) {
    x = 2;
    print (a);
    a = 5;
    return x+a;
}
void main() {
    foo(x); //result?
    print(x); //result?
}
```



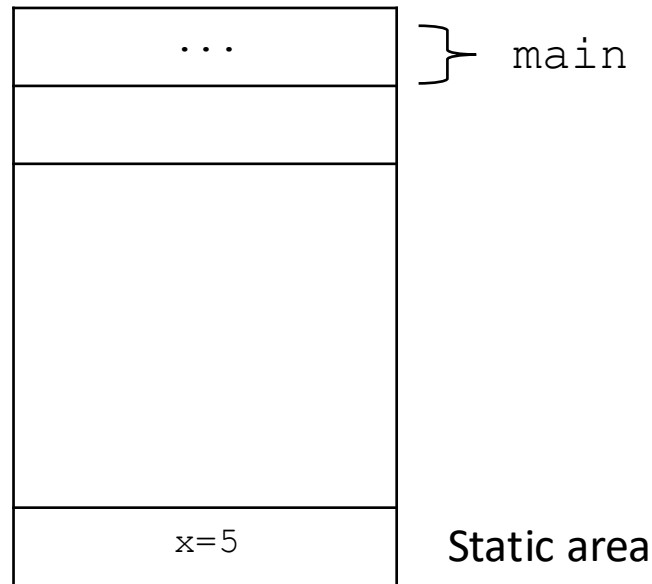
Call-By-Value-Return

```
int x = 1;
int foo (int a) {
    x = 2;
    print (a);
    a = 5;
    return x+a;
}
void main() {
    foo(x); //result?
    print(x); //result?
}
```



Call-By-Value-Return

```
int x = 1;
int foo (int a) {
    x = 2;
    print (a);
    a = 5;
    return x+a;
}
void main() {
    foo(x); //result?
    print(x); //result?
}
```





Call-By-Value-Return

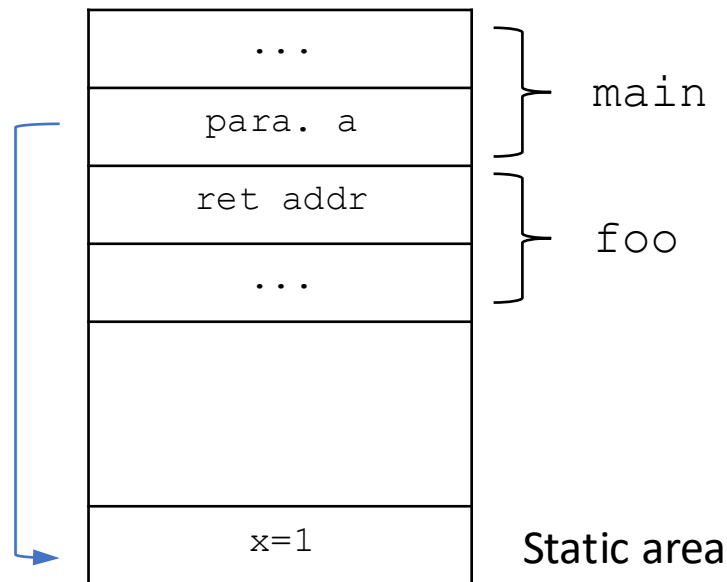
- Characteristics
 - Mostly identical to call-by-value, except an extra step of copying values back to actual parameter

Call-By-Reference

- Calling mechanism
 - Arguments are evaluated to their values
 - Memory or registers allocated for arguments on AR
 - Argument **address** stored in AR
 - AR destroyed when callee returns

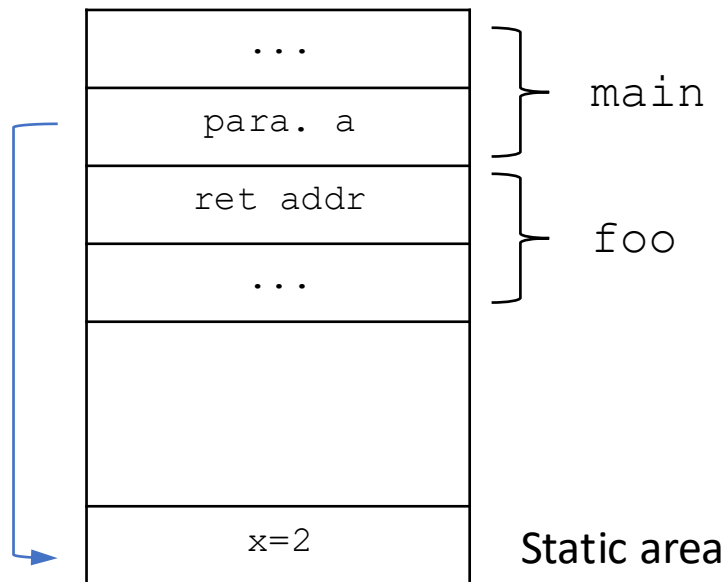
Call-By-Reference

```
int x = 1;
int foo (int a) {
    x = 2;
    print (a);
    a = 5;
    return x+a;
}
void main() {
    foo(x); //result?
    print(x); //result?
}
```



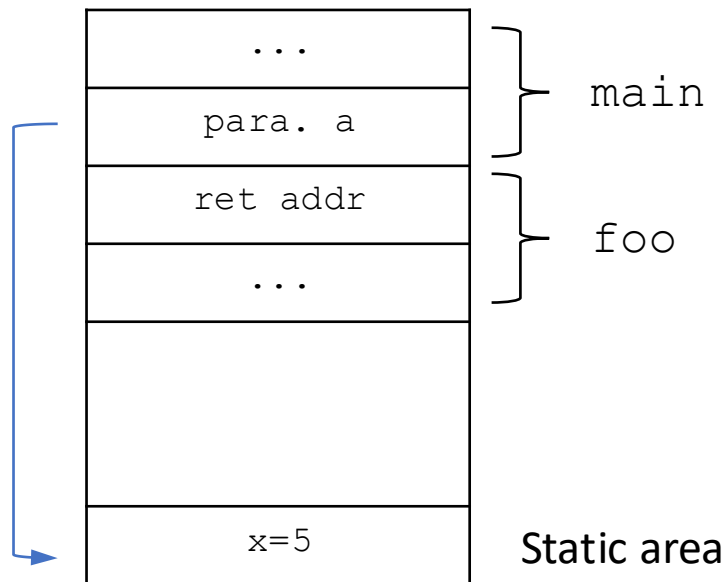
Call-By-Reference

```
int x = 1;
int foo (int a) {
    x = 2;
    print (a);
    a = 5;
    return x+a;
}
void main() {
    foo(x); //result?
    print(x); //result?
}
```



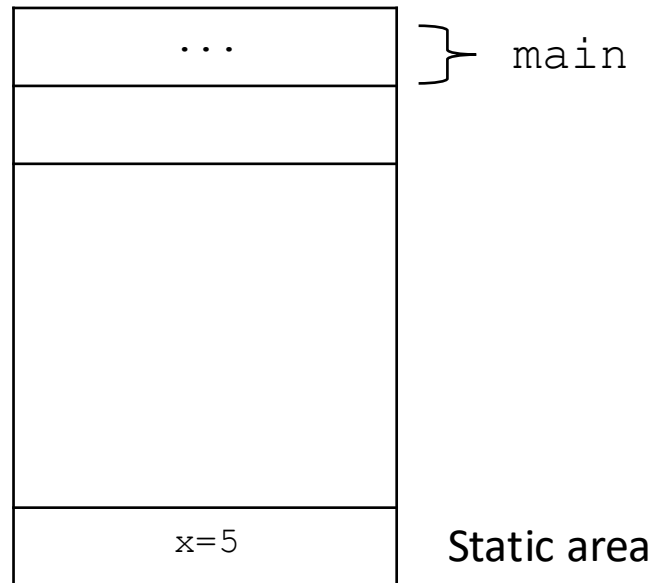
Call-By-Reference

```
int x = 1;
int foo (int a) {
    x = 2;
    print (a);
    a = 5;
    return x+a;
}
void main() {
    foo(x); //result?
    print(x); //result?
}
```



Call-By-Reference

```
int x = 1;
int foo (int a) {
    x = 2;
    print (a);
    a = 5;
    return x+a;
}
void main() {
    foo(x); //result?
    print(x); //result?
}
```



Call-By-Reference

- Formal param is an **alias** of actual param
 - Their values are the same
- Characteristics
 - Actual parameters may directly be changed in callee
 - Some lang disallows complex expressions as arguments
 - in C++, only **l-value** expression could be used
 - Programs are harder to understand (more error-prone)
 - in C++, combine with **const** to make the reference not modifiable to avoid errors



Call-By-Reference: Performance

- Avoids the cost of memory copy
- Indirect memory access: address->value
- C: programmer must pass pointers/references to avoid copying cost
- Java: constructed types use call-by-reference

Key Points

- **Call-by-value**: formal & actual param have **separate memory**: their values may diverge
- **Call-by-value-return**: CBV + **copy** final values of formal param **back** to actual param
- **Call-by-reference**: formal param is an **alias** of actual param: their values are the same

Exercise

```
int x = 1;
int y = 2;
void swap (int a, int b)
{
    int t = a;
    a = b;
    b = t;
}
swap(x, y)
```

- Are the values of x and y swapped?
 - CBV
 - CBVR
 - CBR

Exercise

```
int p = 5;
int q = 2;
int f (int b, int c) {
    b = 2 * c;
    c = 3 + p;
    return b + c;
}
print f(p,q);
print p;
print q;
```

- What is the output?
 - CBV
 - CBVR
 - CBR

Call-By-Name (Lazy Evaluation)

- Calling mechanism
 - Arguments are **not evaluated** to their values (like macros)
 - Actual param replace all formal param in body
 - Still, a useful mode
 - ALGOL 60, Simula

Call-By-Name

```
int A[4] = {1, 2, 3, 4};  
int i = 3;  
void swap (int a, int b)  
{  
    int t = a;  
    a = b;  
    b = t;  
}  
swap(i, A[i])
```

- Value swapped?

Call-By-Name (Lazy Evaluation)

```
function foo (x, y) {  
    return x  $\otimes$  x;  
}  
  
function bar (x) {  
    // long computation  
}  
  
foo(bar(3), bar(4));  
//return bar(3)  $\otimes$  bar(3)
```

- Call-By-Need
 - A memoized variant of CBN
 - Argument is **evaluated** when used for the **first time**, then **stored** for **subsequent use**
 - Haskell, R



Agenda

- Calling sequence
- Parameter passing modes
- Exception handling

Exceptions

- So far, we assumed each function will run from start to its return points
- But a program also needs to handle exceptions:
 - Out of memory
 - Divide by zero
 - File not found
 - ...

Workaround in C language

- Return a special value (e.g., **NULL**) when a real value cannot be computed
- Return an explicit “status” to indicate if an exception happens
- Rely on the caller to pass in the exception handling code (e.g., via function pointer)
- Exception handling is not enforced (error-prone)
- Clutter up the program, especially for normal cases

Exception Handling

- A language feature that
 - Isolates error-checking code
 - Direct execution to a handler when appropriate
- How to define exceptions?
- How does exception change control flow?

Defining Exceptions

- Pre-defined exceptions
 - E.g., divide by zero, out-of-bound array access
- User-defined exceptions
 - Typically, defined as a special kind of class

```
public class myException extends Exception {  
    private int i;  
    public myException(int x) { this.i = x; }  
    public int getI() { return i; }  
}
```

Throwing Exceptions

- Pre-defined exceptions
 - Automatically generated
- User-defined exceptions
 - Generated by **throw** keyword

```
int foo () {  
    ...  
    throw new myException(10);  
    ...  
}
```

Catching Exceptions (Try-Catch Block)



```
try {  
    .. code that might throw exceptions ..  
} catch (Exception1 e1) {  
    .. handling code ..  
} catch (Exception2 e2) {  
    .. handling code ..  
}  
finally { //always executes when exiting block  
    //regardless of if exception occurred or not  
    .. cleanup routine ..  
}
```

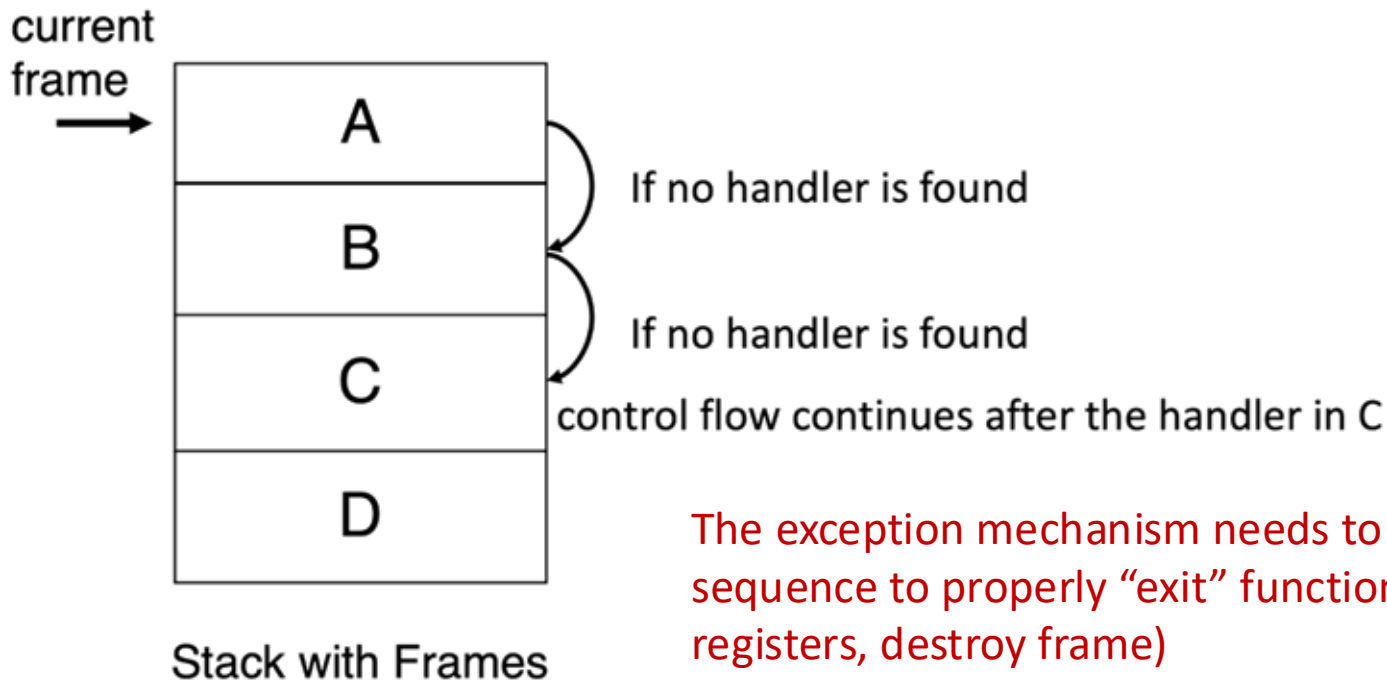
The Replacement Semantics

```
int foo () {  
    ...  
    try {  
        ...  
        throw new myException(10);  
        ... // this is replaced with handling code  
    } catch (myException e) {  
        .. handling code ..  
    }  
    ...  
}
```

The Binding Semantics

```
int foo () {  
    ...  
    try {  
        ...  
        throw new myException(10);  
        ... // resume execution after handling code  
    } catch (myException e) {  
        .. handling code ..  
    }  
    ...  
}
```


Exception Propagation



The exception mechanism needs to follow calling sequence to properly “exit” functions (e.g., restore registers, destroy frame)

Exception Implementation

- When encounter an exception at **pc**:
 - Go through handlers associated with **pc** in sequence
 - Each handler either **executes** (when exception matches) or **re-throws** the exception for next handler
 - If exception not handled, use a function-level handler: **exit the function properly and re-throw the exception**
 - Now, **pc** is in the caller, and repeat the first bullet in the caller

Example (Replacement Semantics)

```
void foo () {  
    try {  
        throw new Exception1();  
        print ("A");  
        throw new Exception2();  
        print ("B");  
    }  
    catch (Exception1 e1) {  
        print "handler1";  
    }  
    print ("C");  
    throw new Exception1();  
}
```

```
void main () {  
    try {  
        try {  
            foo();  
            print ("D");  
        }  
        catch (Exception1 e1) {  
            print "handler2";  
        }  
    }  
    catch (Exception2 e2) {  
        print "handler3";  
    }  
}
```

Example (Binding Semantics)

```
void foo () {  
    try {  
        throw new Exception1();  
        print ("A");  
        throw new Exception2();  
        print ("B");  
    }  
    catch (Exception1 e1) {  
        print "handler1";  
    }  
    print ("C");  
    throw new Exception1();  
}
```

```
void main () {  
    try {  
        try {  
            foo();  
            print ("D");  
        }  
        catch (Exception1 e1) {  
            print "handler2";  
        }  
    }  
    catch (Exception2 e2) {  
        print "handler3";  
    }  
}
```

Catching Exceptions - RAII

- Some languages (e.g., C++, .Net languages like C#)
- Resource is tied to **object lifetime**
 - Resource allocation (or acquisition) is done during object creation, by the **constructor**
 - Resource deallocation (release) is done during object destruction, by the **destructor**
 - The destructor is automatically called when object is going out of scope